

based repositories that either do not use proper dependency management, or copy the source files from the vendor into their repositories.

PHP has its own dependency management system! It's called Composer and it works pretty well. You can easily incorporate it into a project and it will manage your dependencies for you. It will take care of versioning and auto-loading, letting you focus on writing your business logic.

Head over to <https://getcomposer.org/> and install Composer following the instructions on the well documented website. You can install it locally to your project or globally. And if you are going to work on more than one PHP project in your life, installing Composer globally will make your life a bit easier.

Installing packages from Composer is as easy as:

```
composer require twig/twig
```

Composer will generate a *composer.json* and a *composer.lock* file, which will contain all your project configuration and dependency information. A separate directory will be downloaded at your root folder called *vendor/*, which will contain the actual source files of your dependencies.

To use the dependencies, simply load the autoloader, using the following command:

```
require_once "vendor/autoload.php"
```

Then you can use the dependencies by simply importing them in your project files:

```
use Twig\Environment;

$twig = new Environment(...);
```

As you can see here, we do not include individual source files. We simply import them using their name spaces, and the autoloader includes the necessary files for us.

Structuring and name spacing

Composer allows us to define our own name spaces, so we can import our project files in a clean, uniform way without using *include* or *require* statements. In the *composer.json* file, simply add the following *json*:

```
"autoload": {
    "psr-4": {
        "YourProject\\": "src/"
    },
},
```

This allows the autoloader to point to the *src/* folder. So, now you can import the files using the *use* statement like we saw earlier.

Name spacing is done in a way that is similar to how it is done in Java. A file in *src/Controller/Home.php* will be in the *YourProject\Controller* name space. You can import it by using the following command:

```
namespace YourProject;

use YourProject\Controller\Home;
```

You also have to define the name space of the current scope. In the above snippet, the name space is *YourProject*, which means that the file is in the *src/* directory. Similarly, the *Home* class will have a name space as shown below:

```
namespace YourProject\Controller;

class Home { ... }
```

It's a good idea to structure your project in a uniform way that describes its intent and keeps similar things together. Here is a sample directory structure for a good starting point:

```
app/
  • config/
    • db_config.xml
  • cache/
src/
```

```
  • Model/
  • View/
  • Controller/
public/
  • index.php
  • css/
  • js/
  • img/
composer.json
composer.lock
vendor/
```

As you can see, the directories are structured according to their purpose. The *src/* directory contains only PHP source files relevant to your project, while the *public/* directory contains files and assets that are to be accessed by your users.

It's 2019, and combining HTML and PHP together in the same source file with business logic is a very bad practice. Consider using a templating language like Twig. We will discuss templating in the next article in this series.

Dependency injection

Now that we have understood how to separate your concerns, let's take a step further and decouple our code to make it adhere to SOLID principles. The D in SOLID stands for the Dependency Inversion Principle.

Details of the principle will not be discussed here, but we will see how we can implement it on our project. Let's suppose we have a class called *Car*. A car needs wheels, engines, etc, to work. So those are the dependencies of the car. The *Car* class can look something like what's shown below:

```
namespace YourProject\Model;

public class Car
{
    private $engine;
    private $wheels;
    public function __construct()
    {
        $this->engine = new V8Engine();
    }
}
```